EXPERIMENT 13

Case Study on Low-Code Limitations for a Complex Enterprise Use Case

Aim

Evaluate where low-code supports an enterprise insurance process and where a hybrid architecture needs additional engineering.

Record status

Analytical case study completed. The enterprise scenario is hypothetical and is not a production deployment claim.

Concept

Consider a hypothetical insurer coordinating intake, document review, fraud analysis, approvals and bank payouts. Creator can provide forms, reports and controlled business workflows. Requirements such as concurrent payout requests, external reconciliation and long document histories introduce constraints that need explicit tests and service contracts.

Procedure

1. Define the enterprise case: several processing teams share claim records, uploaded documents and an approval trail. Use fictional data and state that transaction volume is an assumption.

2. Map the process into intake, review, decision, payout initiation and reconciliation. Separate the business claim status from the external payment state.

3. Identify failure cases: duplicate submission, two simultaneous payout requests, callback replay, missing surveyor report, wrong user role and delayed provider response.

4. Assess each requirement against platform configuration, custom Deluge and an external service. Prefer a small demonstrable boundary over an unsupported claim of full automation.

5. Recommend a hybrid solution with Creator as the operations interface and a dedicated payout service responsible for provider interaction and reconciliation.

Application design and observations

Requirement / limitation	Case-study response
Complex data relationships	Normalize related forms and review subform growth
External transaction failure	Persist requests and reconcile provider results
Duplicate or concurrent payout	Use a durable unique operation key and verified transaction handling
Large histories and reporting	Archive or separate event history; measure real report performance
Platform coupling	Document schema, business rules and interfaces for migration
Release and governance	Use environment tests, access review and recovery evidence

The earlier laboratory payout demo uses sequential mock checks. It proves selected branches but does not prove bank integration, delayed callbacks or concurrent idempotency. Likewise, a queued email is weaker evidence than confirmed delivery. These concrete boundaries show why a working classroom prototype is only one part of enterprise acceptance.

Validation and discussion

Enterprise acceptance question	Evidence needed
Can duplicate payout requests create two transfers?	Concurrent tests against a payout sandbox
Can a callback be forged or replayed?	Provider authentication and replay tests
Can a processor approve another team's claims?	Independent-account access tests
Can operations recover after failure?	Restore, replay and reconciliation exercise
Can the platform meet the target load?	Measured representative workload

Result

The case study recommends a hybrid architecture. Creator remains suitable for intake, work queues, controlled review and management reports. A dedicated external service should handle bank-provider protocols and durable payout operations. The recommendation is an architectural inference from the stated failure cases, not a claim that low-code always fails at a particular volume.

Viva focus

Explain idempotency, concurrency and why the payout service needs its own acceptance tests.

Reading: Platform performance considerations (official Zoho documentation, reviewed 19 October 2026).

Reading: Get records and pagination (official Zoho documentation, reviewed 19 October 2026).